SISTEMA AGTECH DE EVALUACIÓN DE RINDES AGRÍCOLAS

Integración numérica por los métodos del trapecio y Simpson

Institución: Uniminuto

Programa: Sistema de Evaluación de Rindes Agrícolas por Integración Numérica (Trapecio y Simpson)

Asignatura: Estructura de datos

Estudiante(s): Brayan Bernal Molina

Docente: Edilberto Ramirez Rivera

Fecha: 19/10/2026

1. Introducción

La agricultura de precisión requiere estimaciones confiables del rendimiento de los cultivos para comparar sectores de una parcela, planear cosechas y apoyar decisiones de manejo. Las mediciones de campo suelen obtenerse en puntos separados; por ello, la integral del perfil de rendimiento puede aproximarse mediante métodos numéricos.

Este proyecto propone una aplicación de escritorio en Java Swing que compara las reglas compuestas del trapecio y Simpson 1/3, organiza mediciones y resultados usando las estructuras de datos exigidas y presenta una tabla de resultados. Los datos incluidos son demostrativos y deben reemplazarse por mediciones verificadas antes de cualquier uso agrícola real.

2. Objetivos

Objetivo general: diseñar una aplicación Java con GUI que estime y compare integrales de una curva de rendimiento agrícola mediante trapecio y Simpson 1/3.

- Almacenar las posiciones y rendimientos mediante arreglos unidimensionales.
- Comparar métodos y configuraciones mediante una matriz bidimensional.
- Usar listas simple, doble y circular para historiales, subparcelas y perímetro.
- Integrar una pila LIFO para intervalos pendientes de refinamiento y una cola FIFO para resultados parciales.
- Explicar la metodología matemática, las estructuras y su complejidad computacional.

3. Marco teórico

3.1 Estructuras de datos obligatorias

Estructura	Uso en el aplicativo	Operación principal
Vector 1D	Posiciones x y rendimientos y	Acceso por índice O(1)
Matriz 2D	Comparación de método y número de subintervalos	Acceso por índice O(1)
Lista simple	Historial de diferencias absolutas	Insertar al final O(n); recorrer O(n)
Lista doble	Subparcelas contiguas; enlace anterior/siguiente	Recorrido O(n)
Lista circular	Cuatro vértices del perímetro conectados en ciclo	Recorrido O(n)
Pila LIFO	Intervalos que podrían refinarse	push/pop O(1)
Cola FIFO	Resultados parciales por franja	offer/poll O(1) amortizado

4. Diseño y arquitectura

La clase principal SistemaRindesAgTech hereda de JFrame y contiene la interfaz Swing. La GUI captura la longitud y un rendimiento de referencia; permite cargar el perfil de ejemplo, calcular los dos métodos, consultar resultados en una tabla y limpiar la salida.

El cálculo usa los vectores de posiciones y rendimientos. Los resultados se guardan en la matriz comparativa. La diferencia se agrega a la lista simple; las franjas se modelan como subparcelas en una lista doble; los vértices se enlazan circularmente; los intervalos se apilan para representar refinamiento y las contribuciones parciales pasan por una cola FIFO.

5. Metodología de desarrollo

1. Identificar las variables: posición espacial y rendimiento observado.
2. Definir un perfil de mediciones de ejemplo para verificar el funcionamiento.
3. Implementar los métodos de integración compuesta y comprobar que Simpson reciba un número par de subintervalos y espaciamiento uniforme.
4. Integrar las estructuras de datos y describir su función concreta.
5. Construir la GUI con Java Swing, validación de entradas y tabla de resultados.
6. Probar entradas válidas, valores negativos, campos no numéricos y el botón de limpieza.

6. Ejemplo de resultados

Perfil de demostración: posiciones 0, 10, 20, 30, 40, 50 y 60 m; rendimientos 2.0, 2.8, 3.5, 4.1, 3.8, 3.2 y 2.9 ton/ha. Son seis subintervalos uniformes, por lo que se cumplen las condiciones de Simpson 1/3 compuesto. La aplicación muestra los valores calculados y la diferencia absoluta.

Interpretación: la integral combina el rendimiento por unidad de superficie con la dimensión espacial muestreada. Para calcular toneladas totales se requiere el ancho real de la franja o área muestreada y convertir correctamente las unidades. No debe interpretarse directamente la integral como toneladas sin esa información.

7. Complejidad Big-O

Para n puntos de muestreo, trapecio y Simpson recorren los puntos una vez: O(n) tiempo y O(1) memoria adicional por cálculo. La creación de subparcelas y el procesamiento de franjas son O(n). Acceder a una posición de los vectores y matriz es O(1). Insertar un error al final de la lista simple, con búsqueda desde la cabeza, cuesta O(n). Las operaciones básicas de pila y cola son O(1) amortizado. El almacenamiento total del perfil es O(n).

9. Conclusiones

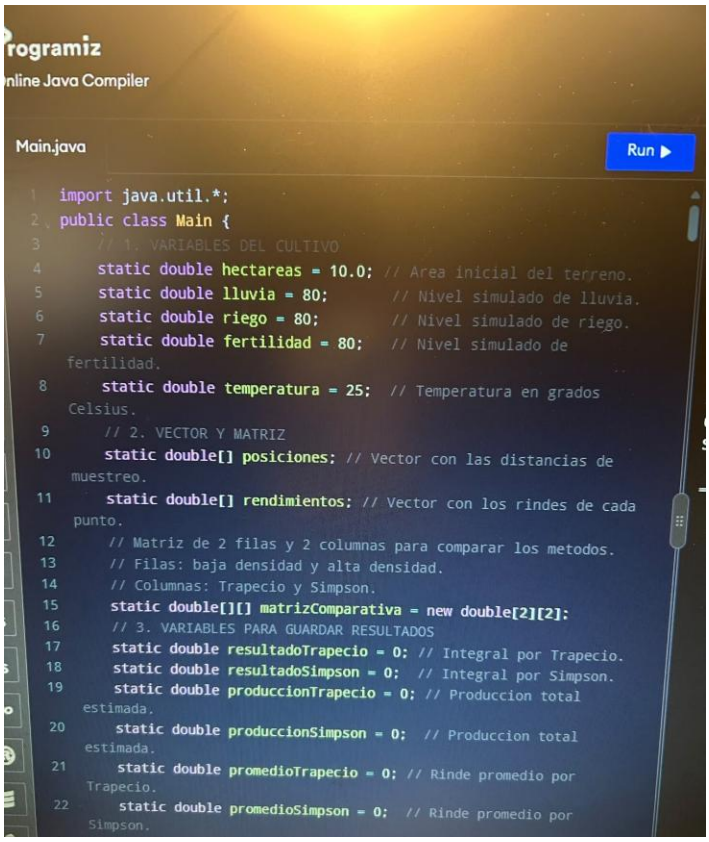
El proyecto muestra cómo la integración numérica y las estructuras de datos pueden combinarse para apoyar el análisis de rendimiento agrícola. Trapecio es sencillo y útil para datos discretos; Simpson puede ofrecer mayor precisión bajo sus condiciones de aplicación. Las listas, la pila y la cola representan historiales y procesos distintos, mientras que la GUI facilita el uso por parte del estudiante. Para llevar el prototipo a una finca se necesitan mediciones reales, metadatos de área y validación agronómica.

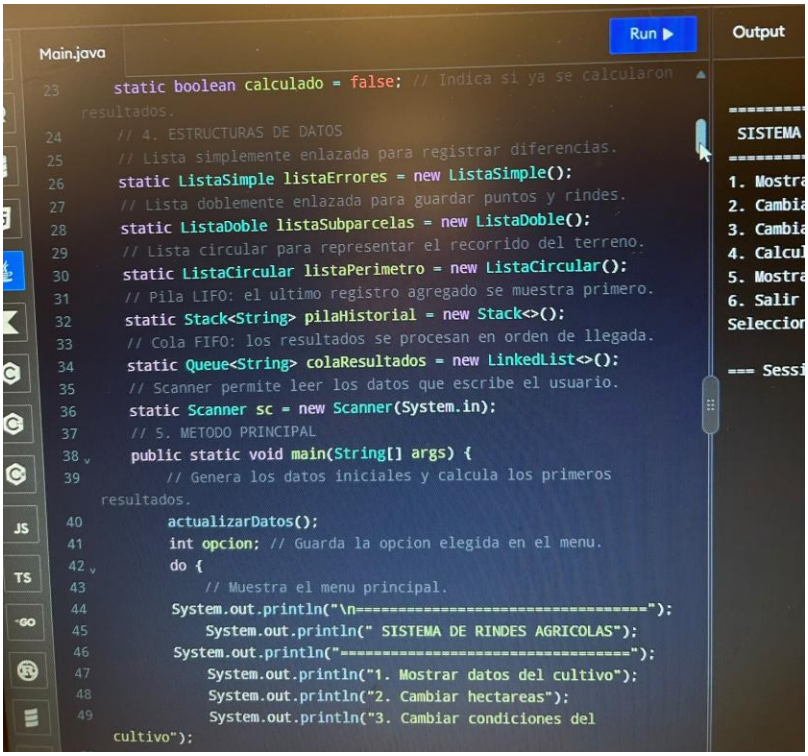
10. Repositorio y Link del codigo

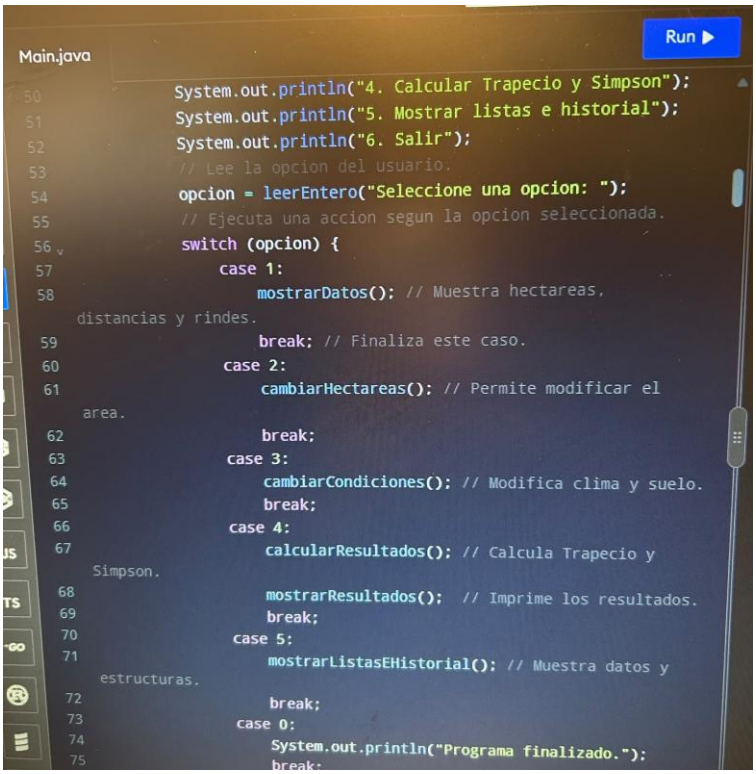
link del codigo
https://www.programiz.com/online-compiler/5vpVEySJbanmy

Link GIT HUB
https://github.com/brayan-bm/estructura-de-datos-.git

11. Fotos del GUI

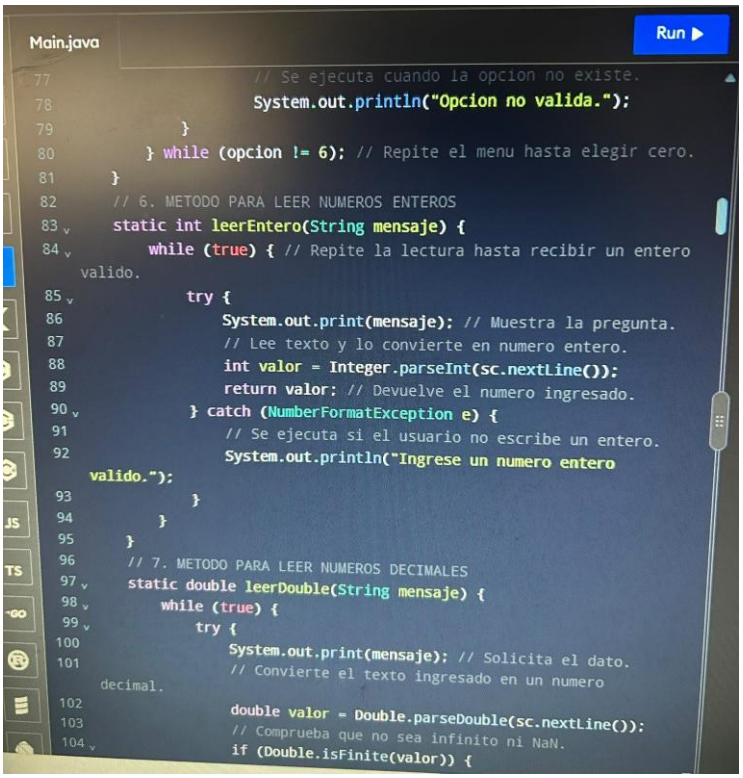






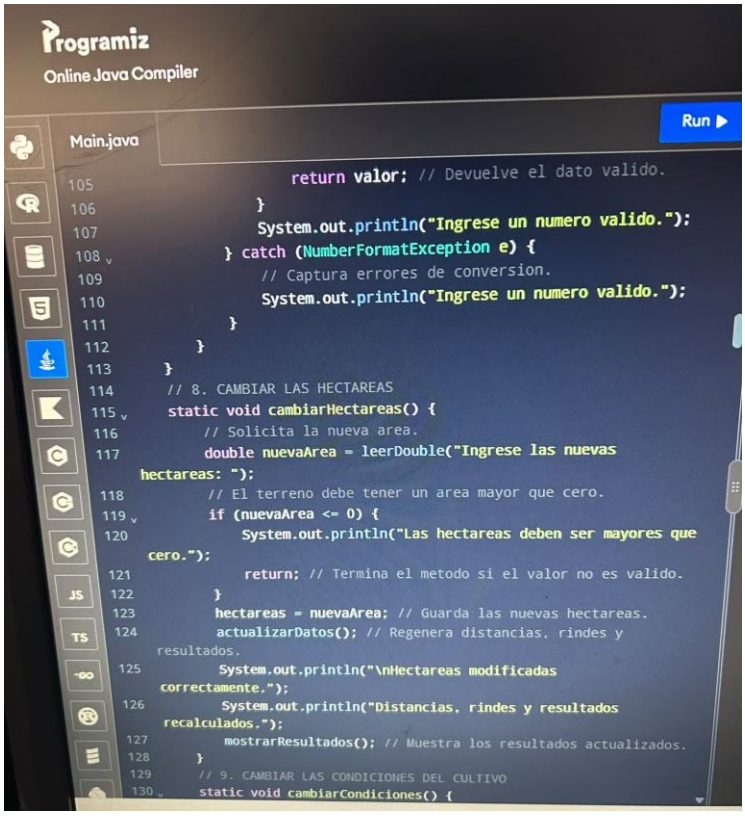
```
Main.java                                                    Run ▶

            System.out.println("4. Calcular Trapecio y Simpson");
            System.out.println("5. Mostrar listas e historial");
            System.out.println("6. Salir");
            // Lee la opcion del usuario.
            opcion = leerEntero("Seleccione una opcion: ");
            // Ejecuta una accion segun la opcion seleccionada.
            switch (opcion) {
                case 1:
                    mostrarDatos(); // Muestra hectareas,
    distancias y rindes.
                    break; // Finaliza este caso.
                case 2:
                    cambiarHectareas(); // Permite modificar el
    area.
                    break;
                case 3:
                    cambiarCondiciones(); // Modifica clima y suelo.
                    break;
                case 4:
                    calcularResultados(); // Calcula Trapecio y
    Simpson.
                    mostrarResultados();  // Imprime los resultados.
                    break;
                case 5:
                    mostrarListasEHistorial(); // Muestra datos y
    estructuras.
                    break;
                case 0:
                    System.out.println("Programa finalizado.");
                    break;
```



```
Main.java                                                    Run ▶

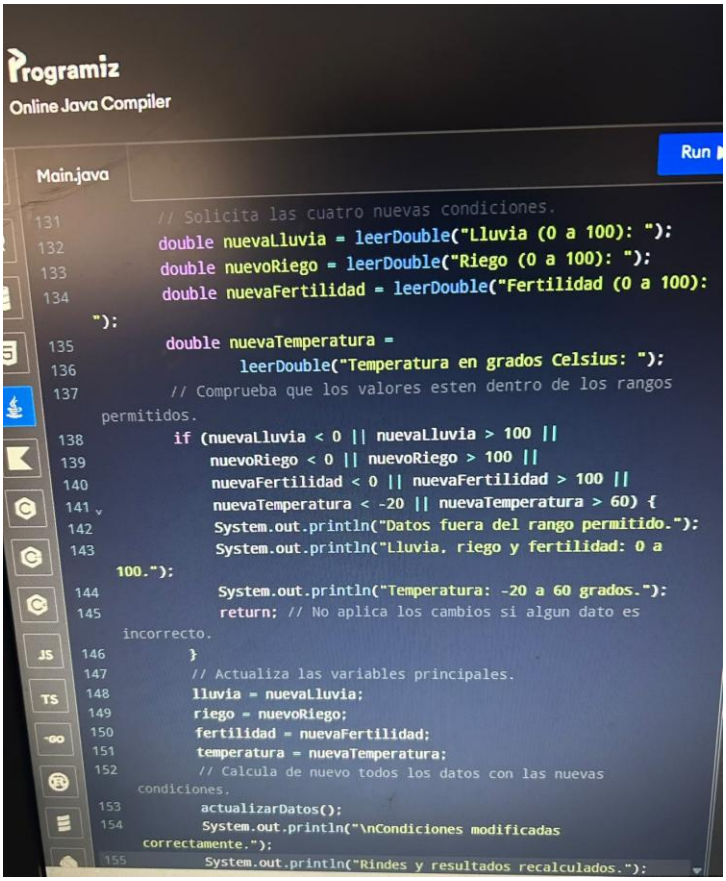
                // Se ejecuta cuando la opcion no existe.
                System.out.println("Opcion no valida.");
            }
        } while (opcion != 6); // Repite el menu hasta elegir cero.
    }
    // 6. METODO PARA LEER NUMEROS ENTEROS
    static int leerEntero(String mensaje) {
        while (true) { // Repite la lectura hasta recibir un entero
valido.
            try {
                System.out.print(mensaje); // Muestra la pregunta.
                // Lee texto y lo convierte en numero entero.
                int valor = Integer.parseInt(sc.nextLine());
                return valor; // Devuelve el numero ingresado.
            } catch (NumberFormatException e) {
                // Se ejecuta si el usuario no escribe un entero.
                System.out.println("Ingrese un numero entero
valido.");
            }
        }
    }
    // 7. METODO PARA LEER NUMEROS DECIMALES
    static double leerDouble(String mensaje) {
        while (true) {
            try {
                System.out.print(mensaje); // Solicita el dato.
                // Convierte el texto ingresado en un numero
decimal.
                double valor = Double.parseDouble(sc.nextLine());
                // Comprueba que no sea infinito ni NaN.
                if (Double.isFinite(valor)) {
```



Programiz
Online Java Compiler

```
Main.java                                                    Run ▶

                    return valor; // Devuelve el dato valido.
                }
                System.out.println("Ingrese un numero valido.");
            } catch (NumberFormatException e) {
                // Captura errores de conversion.
                System.out.println("Ingrese un numero valido.");
            }
        }
    }
    // 8. CAMBIAR LAS HECTAREAS
    static void cambiarHectareas() {
        // Solicita la nueva area.
        double nuevaArea = leerDouble("Ingrese las nuevas
hectareas: ");
        // El terreno debe tener un area mayor que cero.
        if (nuevaArea <= 0) {
            System.out.println("Las hectareas deben ser mayores que
cero.");
            return; // Termina el metodo si el valor no es valido.
        }
        hectareas = nuevaArea; // Guarda las nuevas hectareas.
        actualizarDatos(); // Regenera distancias, rindes y
resultados.
        System.out.println("\nHectareas modificadas
correctamente.");
        System.out.println("Distancias, rindes y resultados
recalculados.");
        mostrarResultados(); // Muestra los resultados actualizados.
    }
    // 9. CAMBIAR LAS CONDICIONES DEL CULTIVO
    static void cambiarCondiciones() {
```



Programiz
Online Java Compiler

```
Main.java                                                    Run ▶

        // Solicita las cuatro nuevas condiciones.
        double nuevaLluvia = leerDouble("Lluvia (0 a 100): ");
        double nuevoRiego = leerDouble("Riego (0 a 100): ");
        double nuevaFertilidad = leerDouble("Fertilidad (0 a 100):
");
        double nuevaTemperatura =
                leerDouble("Temperatura en grados Celsius: ");
        // Comprueba que los valores esten dentro de los rangos
permitidos.
        if (nuevaLluvia < 0 || nuevaLluvia > 100 ||
            nuevoRiego < 0 || nuevoRiego > 100 ||
            nuevaFertilidad < 0 || nuevaFertilidad > 100 ||
            nuevaTemperatura < -20 || nuevaTemperatura > 60) {
            System.out.println("Datos fuera del rango permitido.");
            System.out.println("Lluvia, riego y fertilidad: 0 a
100.");
            System.out.println("Temperatura: -20 a 60 grados.");
            return; // No aplica los cambios si algun dato es
incorrecto.
        }
        // Actualiza las variables principales.
        lluvia = nuevaLluvia;
        riego = nuevoRiego;
        fertilidad = nuevaFertilidad;
        temperatura = nuevaTemperatura;
        // Calcula de nuevo todos los datos con las nuevas
condiciones.
        actualizarDatos();
        System.out.println("\nCondiciones modificadas
correctamente.");
        System.out.println("Rindes y resultados recalculados.");
```

Programiz
Online Java Compiler

Main.java

```
        mostrarResultados(); // Presenta los nuevos resultados.
    }
    // 10. ACTUALIZAR TODO EL SISTEMA
    static void actualizarDatos() {
        generarDatos(); // Actualiza los vectores y las listas.
        calcularResultados(); // Recalcula las integrales y la
produccion.
        guardarHistorial(); // Registra la configuracion actual.
    }
    // 11. GENERAR DISTANCIAS Y RENDIMIENTOS
    static void generarDatos() {
        // Calcula los intervalos segun las hectareas.
        // Math.ceil redondea hacia arriba.
        // Math.max garantiza un minimo de cuatro intervalos.
        int intervalos = Math.max(
                4, (int) Math.ceil(hectareas / 2.0)
        );
        // Simpson necesita un numero par de intervalos.
        if (intervalos % 2 != 0) {
            intervalos++; // Aumenta uno si el numero es impar.
        }
        // Para N intervalos se necesitan N + 1 puntos.
        posiciones = new double[intervalos + 1];
        rendimientos = new double[intervalos + 1];
        // Convierte las hectareas a metros cuadrados.
        // Se supone que el terreno tiene forma cuadrada.
        double longitud = Math.sqrt(hectareas * 10000.0);
        // Recorre todos los puntos de muestreo.
        for (int i = 0; i <= intervalos; i++) {
            // Calcula la posicion relativa entre cero y uno.
            double fraccion = (double) i / intervalos;
```

Main.java

```
            // Calcula la distancia en metros desde el inicio.
            posiciones[i] = fraccion * longitud;
            // Calcula el rinde usando las condiciones actuales.
            rendimientos[i] = calcularRinde(fraccion);
        }
        // Crea listas nuevas para reflejar los datos actuales.
        listaErrores = new ListaSimple();
        listaSubparcelas = new ListaDoble();
        listaPerimetro = new ListaCircular();
        // Guarda cada punto y su rendimiento en la lista doble.
        for (int i = 0; i < rendimientos.length; i++) {
            listaSubparcelas.agregar(
                    "Punto " + (i + 1)
                    + " - Rinde: "
                    + String.format("%.2f", rendimientos[i])
                    + " ton/ha"
            );
        }
        // Agrega las etapas del recorrido del perimetro.
        listaPerimetro.agregar("Inicio del terreno");
        listaPerimetro.agregar("Lado derecho");
        listaPerimetro.agregar("Parte posterior");
        listaPerimetro.agregar("Lado izquierdo");
        listaPerimetro.agregar("Regreso al inicio");
    }
    // 12. FORMULA SIMULADA DEL RINDE AGRICOLA
    static double calcularRinde(double posicion) {
        // Combina lluvia y riego:
        // 40 % del peso corresponde a lluvia y 60 % a riego.
        double agua = (lluvia * 0.4 + riego * 0.6) / 100.0;
```

Programiz
Online Java Compiler

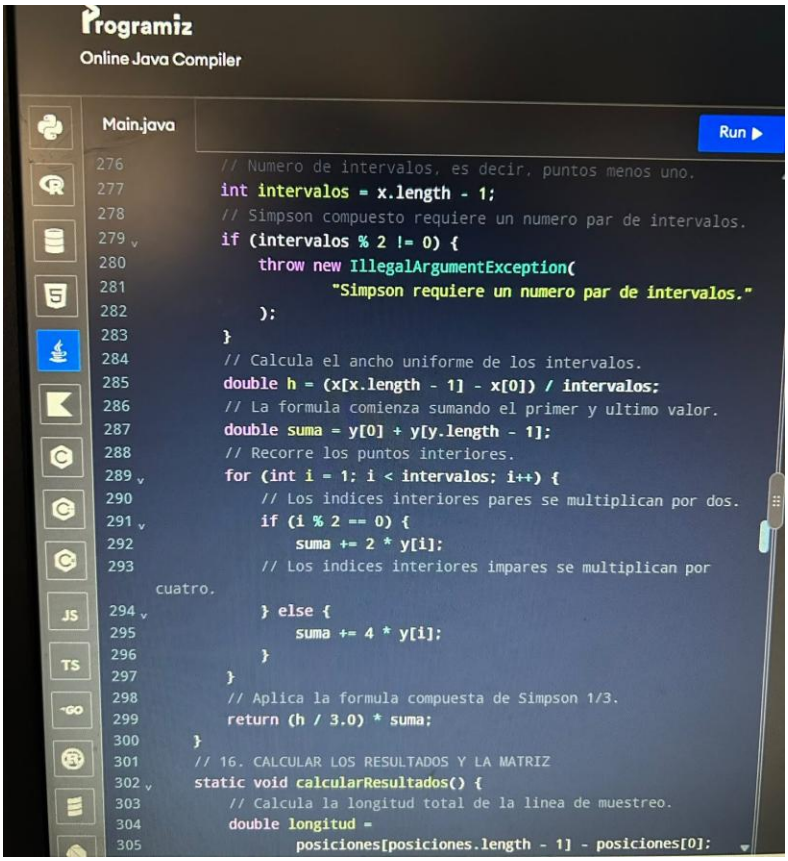
Main.java

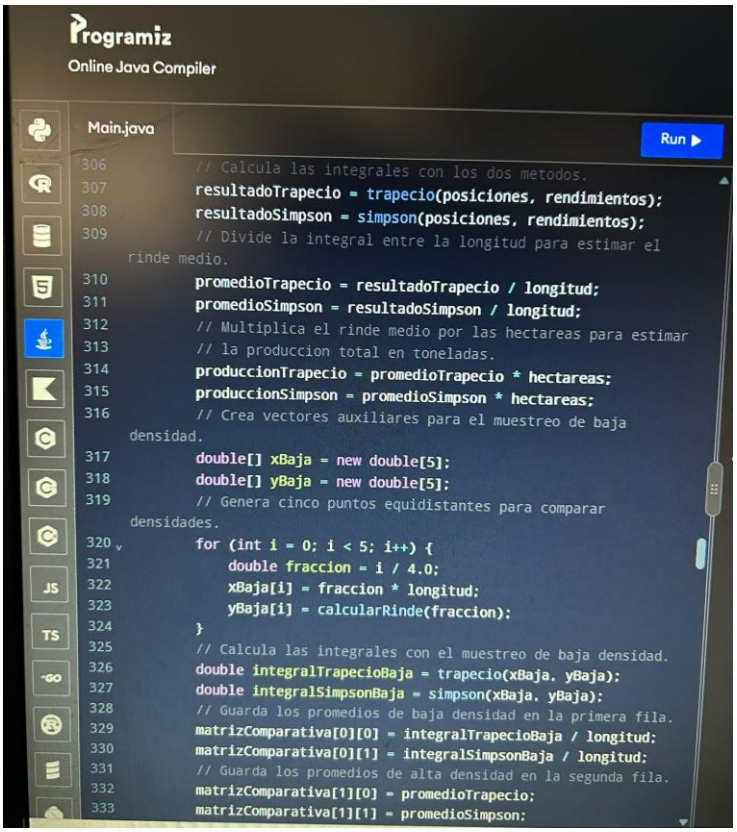
```
        // Convierte la fertilidad porcentual a una escala de cero
a uno.
        double suelo = fertilidad / 100.0;
        // Calcula un factor de temperatura.
        // La referencia del modelo es 25 grados.
        // Math.abs calcula la diferencia absoluta.
        // Math.max impide que el factor sea negativo.
        double factorTemperatura =
                Math.max(
                    0.0,
                    1.0 - Math.abs(temperatura - 25.0) / 45.0
                );
        // Simula variaciones del rinde a lo largo del terreno.
        // Math.sin representa una variacion suave entre puntos.
        double variacion =
                0.90 + 0.10 * Math.sin(Math.PI * posicion);
        // Introduce un efecto simulado del tamaño del area.
        double factorArea =
                0.90 + 0.20 * (hectareas / (hectareas + 10.0));
        // Multiplica los factores para estimar el rinde en ton/ha.
        return 8.0
                * (0.30 + 0.70 * agua)
                * (0.30 + 0.70 * suelo)
                * factorTemperatura
                * variacion
                * factorArea;
    }
    // 13. MOSTRAR LOS DATOS DEL CULTIVO
    static void mostrarDatos() {
        System.out.println("\n===== DATOS ACTUALES =====");
        // Muestra los valores actuales de las variables.
```

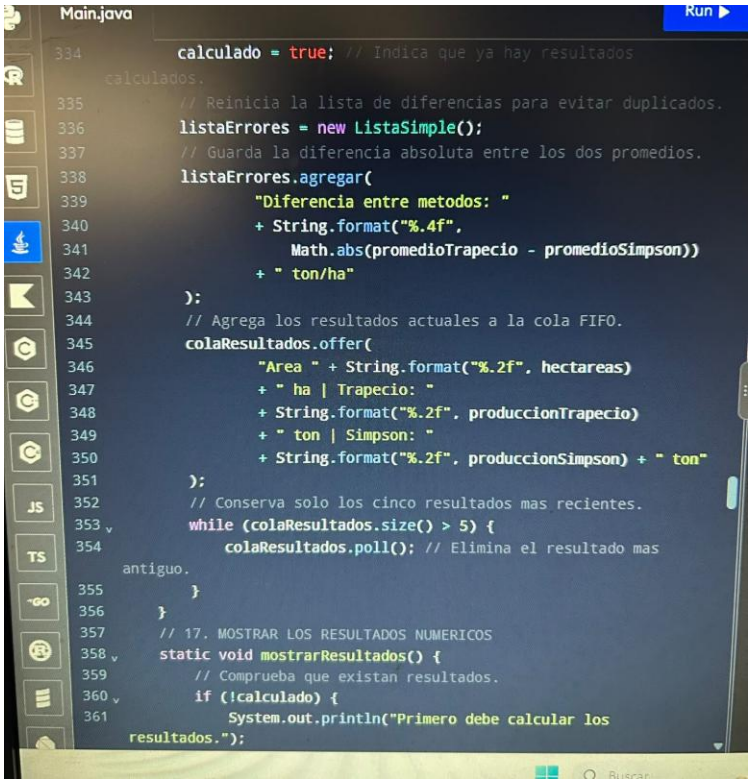
Programiz
Online Java Compiler

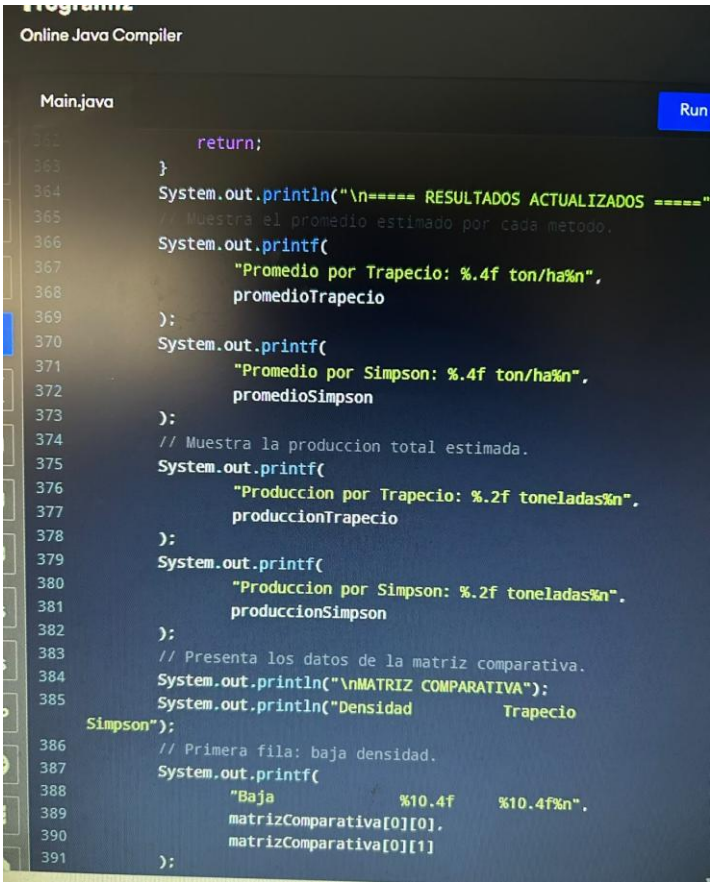
Main.java

```
        System.out.printf("Hectareas: %.2f%n", hectareas);
        System.out.printf("Lluvia: %.2f%%%n", lluvia);
        System.out.printf("Riego: %.2f%%%n", riego);
        System.out.printf("Fertilidad: %.2f%%%n", fertilidad);
        System.out.printf("Temperatura: %.2f C%n", temperatura);
        // Encabezados de la tabla.
        System.out.println("\nPunto | Distancia (m) | Rinde
(ton/ha)");
        // Recorre los vectores y muestra sus valores.
        for (int i = 0; i < posiciones.length; i++) {
            // %d muestra un entero y %.2f muestra dos decimales.
            System.out.printf(
                    "%5d | %13.2f | %13.2f%n",
                    i + 1, posiciones[i], rendimientos[i]
            );
        }
    }
    // 14. METODO DE INTEGRACION DEL TRAPECIO
    static double trapecio(double[] x, double[] y) {
        double suma = 0; // Acumula las areas de los trapecios.
        // Recorre cada par de puntos consecutivos.
        for (int i = 0; i < x.length - 1; i++) {
            // Calcula la distancia entre dos puntos.
            double distancia = x[i + 1] - x[i];
            // Calcula el area del trapecio y la suma al acumulador.
            suma += distancia * (y[i] + y[i + 1]) / 2.0;
        }
        return suma; // Devuelve la integral aproximada.
    }
    // 15. METODO DE INTEGRACION DE SIMPSON
    static double simpson(double[] x, double[] y) {
```


Programiz
Online Java Compiler
Main.java
Run
// Numero de intervalos, es decir, puntos menos uno.
int intervalos = x.length - 1;
// Simpson compuesto requiere un numero par de intervalos.
if (intervalos % 2 != 0) {
throw new IllegalArgumentException(
"Simpson requiere un numero par de intervalos."
);
}
// Calcula el ancho uniforme de los intervalos.
double h = (x[x.length - 1] - x[0]) / intervalos;
// La formula comienza sumando el primer y ultimo valor.
double suma = y[0] + y[y.length - 1];
// Recorre los puntos interiores.
for (int i = 1; i < intervalos; i++) {
// Los indices interiores pares se multiplican por dos.
if (i % 2 == 0) {
suma += 2 * y[i];
// Los indices interiores impares se multiplican por cuatro.
} else {
suma += 4 * y[i];
}
}
// Aplica la formula compuesta de Simpson 1/3.
return (h / 3.0) * suma;
}
// 16. CALCULAR LOS RESULTADOS Y LA MATRIZ
static void calcularResultados() {
// Calcula la longitud total de la linea de muestreo.
double longitud =
posiciones[posiciones.length - 1] - posiciones[0];


Programiz
Online Java Compiler
Main.java
Run
// Calcula las integrales con los dos metodos.
resultadoTrapecio = trapecio(posiciones, rendimientos);
resultadoSimpson = simpson(posiciones, rendimientos);
// Divide la integral entre la longitud para estimar el rinde medio.
promedioTrapecio = resultadoTrapecio / longitud;
promedioSimpson = resultadoSimpson / longitud;
// Multiplica el rinde medio por las hectareas para estimar
// la produccion total en toneladas.
produccionTrapecio = promedioTrapecio * hectareas;
produccionSimpson = promedioSimpson * hectareas;
// Crea vectores auxiliares para el muestreo de baja densidad.
double[] xBaja = new double[5];
double[] yBaja = new double[5];
// Genera cinco puntos equidistantes para comparar densidades.
for (int i = 0; i < 5; i++) {
double fraccion = i / 4.0;
xBaja[i] = fraccion * longitud;
yBaja[i] = calcularRinde(fraccion);
}
// Calcula las integrales con el muestreo de baja densidad.
double integralTrapecioBaja = trapecio(xBaja, yBaja);
double integralSimpsonBaja = simpson(xBaja, yBaja);
// Guarda los promedios de baja densidad en la primera fila.
matrizComparativa[0][0] = integralTrapecioBaja / longitud;
matrizComparativa[0][1] = integralSimpsonBaja / longitud;
// Guarda los promedios de alta densidad en la segunda fila.
matrizComparativa[1][0] = promedioTrapecio;
matrizComparativa[1][1] = promedioSimpson;


Main.java
Run
calculado = true; // Indica que ya hay resultados calculados.
// Reinicia la lista de diferencias para evitar duplicados.
listaErrores = new ListaSimple();
// Guarda la diferencia absoluta entre los dos promedios.
listaErrores.agregar(
"Diferencia entre metodos: "
+ String.format("%.4f",
Math.abs(promedioTrapecio - promedioSimpson))
+ " ton/ha"
);
// Agrega los resultados actuales a la cola FIFO.
colaResultados.offer(
"Area " + String.format("%.2f", hectareas)
+ " ha | Trapecio: "
+ String.format("%.2f", produccionTrapecio)
+ " ton | Simpson: "
+ String.format("%.2f", produccionSimpson) + " ton"
);
// Conserva solo los cinco resultados mas recientes.
while (colaResultados.size() > 5) {
colaResultados.poll(); // Elimina el resultado mas antiguo.
}
}
// 17. MOSTRAR LOS RESULTADOS NUMERICOS
static void mostrarResultados() {
// Comprueba que existan resultados.
if (!calculado) {
System.out.println("Primero debe calcular los resultados.");


Online Java Compiler
Main.java
Run
return;
}
System.out.println("\n===== RESULTADOS ACTUALIZADOS =====");
// Muestra el promedio estimado por cada metodo.
System.out.printf(
"Promedio por Trapecio: %.4f ton/ha%n",
promedioTrapecio
);
System.out.printf(
"Promedio por Simpson: %.4f ton/ha%n",
promedioSimpson
);
// Muestra la produccion total estimada.
System.out.printf(
"Produccion por Trapecio: %.2f toneladas%n",
produccionTrapecio
);
System.out.printf(
"Produccion por Simpson: %.2f toneladas%n",
produccionSimpson
);
// Presenta los datos de la matriz comparativa.
System.out.println("\nMATRIZ COMPARATIVA");
System.out.println("Densidad Trapecio Simpson");
// Primera fila: baja densidad.
System.out.printf(
"Baja %10.4f %10.4f%n",
matrizComparativa[0][0],
matrizComparativa[0][1]
);

Programiz
Online Java Compiler

Main.java

Run

```
392         // Segunda fila: alta densidad.
393         System.out.printf(
394                 "Alta              %10.4f     %10.4f%n",
395                 matrizComparativa[1][0],
396                 matrizComparativa[1][1]
397         );
398     }
399     // 18. GUARDAR LOS CAMBIOS EN LA PILA
400     static void guardarHistorial() {
401         // Crea un texto con las condiciones y la produccion actual.
402         String registro =
403                 "Area: " + String.format("%.2f", hectareas)
404                 + " ha, lluvia: " + lluvia
405                 + "%, riego: " + riego
406                 + "%, fertilidad: " + fertilidad
407                 + "%, temperatura: " + temperatura
408                 + " C, produccion Simpson: "
409                 + String.format("%.2f", produccionSimpson)
410                 + " toneladas.";
411         // Guarda el registro en la pila LIFO.
412         pilaHistorial.push(registro);
413     }
414     // 19. MOSTRAR LISTAS, DATOS E HISTORIAL
415     static void mostrarListasEHistorial() {
416         System.out.println("\n================================");
417         System.out.println(" LISTAS E HISTORIAL ACTUALIZADOS");
418         System.out.println("================================");
419         // Muestra los valores y vectores actuales.
420         mostrarDatos();
421         // Muestra los ultimos resultados numericos.
422         mostrarResultados();
```

Programiz
Online Java Compiler

Main.java

Run

```
424         System.out.println("\n--- LISTA SIMPLEMENTE ENLAZADA ---");
425         listaErrores.mostrar();
426         // Presenta la lista doblemente enlazada.
427         System.out.println("\n--- LISTA DOBLEMENTE ENLAZADA ---");
428         listaSubparcelas.mostrar();
429         // Presenta la lista circular del perimetro.
430         System.out.println("\n--- LISTA CIRCULAR DEL TERRENO ---");
431         listaPerimetro.mostrar();
432         // Presenta la pila desde el registro mas reciente.
433         System.out.println("\n--- PILA: HISTORIAL DE CAMBIOS ---");
434         if (pilaHistorial.empty()) {
435             System.out.println("No hay operaciones registradas.");
436         } else {
437             // Recorre la pila desde el ultimo registro hasta el
    primero.
438             for (int i = pilaHistorial.size() - 1; i >= 0; i--) {
439                 System.out.println(pilaHistorial.get(i));
440             }
441         }
442         // Presenta los resultados guardados en la cola FIFO.
443         System.out.println("\n--- COLA: ULTIMOS RESULTADOS ---");
444         if (colaResultados.isEmpty()) {
445             System.out.println("No hay resultados registrados.");
446         } else {
447             // Recorre la cola en el orden de llegada.
448             for (String resultado : colaResultados) {
449                 System.out.println(resultado);
450             }
451         }
452     }
```

Output

```
================================
 SISTEMA DE RINDES AGRICOLAS
================================
1. Mostrar datos del cultivo
2. Cambiar hectareas
3. Cambiar condiciones del cultivo
4. Calcular Trapecio y Simpson
5. Mostrar listas e historial
6. Salir
Seleccione una opcion:

=== Session Ended Due To Inactivity ===
```

Referencias sugeridas

Chapra, S. C., & Canale, R. P. Métodos numéricos para ingenieros. McGraw-Hill.

Oracle. Java Swing documentation. https://docs.oracle.com/javase/tutorial/uiswing/

Burden, R. L., & Faires, J. D. Análisis numérico. Cengage Learning.